Módulo 3 · Depuración Avanzada con IA

Refactorización de Código Heredado: Modularización + Tests + Pipeline CI/CD + Documentación

Ejercicio de Clase · Ejercicio 3 de 3 · Proyecto profesional completo con IA como copiloto

Nombre del alumno/a	
Fecha	
🔗 Enlace GitHub del repositorio	

📌 *Instrucciones: En este ejercicio partirás de un código heredado (legacy) real con errores de diseño. Tu misión es refactorizarlo en módulos, escribir tests, configurar un pipeline CI/CD, y documentar cada paso como haría un profesional. Usa la IA como copiloto en cada fase.*

BLOQUE 1 — El Código Heredado

A continuación tienes el código de una calculadora de costes de APIs de IA. Está funcional pero mezcla responsabilidades, carece de tests y no sigue buenas prácticas. Tu trabajo en este ejercicio es transformarlo en un proyecto profesional.

```
# calculadora_costes.py — CÓDIGO HEREDADO

from openai import OpenAI
import tiktoken

PRECIOS = {
    'gpt-4o': {'input': 2.50, 'output': 10.00},
    'gpt-4o-mini': {'input': 0.15, 'output': 0.60},
    'claude-3-sonnet': {'input': 3.00, 'output': 15.00},
    'gemini-1.5-flash': {'input': 0.075, 'output': 0.30},
}

class CalculadoraCostes:
    def __init__(self, modelo: str = 'gpt-4o-mini'):
        self.modelo = modelo
        self.precios = PRECIOS.get(modelo, {'input': 0.15, 'output': 0.60})

    def calcular_costes(self, tokens_input, tokens_output):
        coste_input = (tokens_input / 1_000_000) * self.precios['input']
        coste_output = (tokens_output / 1_000_000) * self.precios['output']
        return {'total': coste_input + coste_output,
                'input': coste_input, 'output': coste_output}

    def estimar_tokens(self, texto: str) -> int:
```

```
        encoding = tiktoken.encoding_for_model(self.modelo)
        return len(encoding.encode(texto))

    def proyectar_uso_mensual(self, llamadas_dia, tok_in, tok_out, dias=30):
        llamadas = llamadas_dia * dias
        coste = self.calcular_costes(llamadas * tok_in, llamadas * tok_out)
        return {'llamadas': llamadas, 'coste': coste['total']}

# --- Ejecución directa (mezclada con la lógica) ---
calc = CalculadoraCostes('gpt-4o-mini')
print(calc.calcular_costes(200, 300))
print(calc.proyectar_uso_mensual(100, 200, 300))
```

1.1 — Identifica los problemas de diseño

Sin refactorizar todavía, lista al menos 5 problemas de diseño del código (no errores de ejecución, sino malas prácticas):

#	Problema detectado	Impacto en producción

BLOQUE 2 — Modularización

🔗 *Conexión con el módulo anterior: tras analizar el código heredado, el primer paso profesional es separarlo en módulos con responsabilidad única (Principio de Responsabilidad Única - SRP).*

2.1 — Diseña la estructura de carpetas

Define la estructura de directorios del proyecto. Indica qué archivo va en cada carpeta y por qué:

Archivo / Carpeta	Responsabilidad única	Funciones que contiene
calculadora/precios.py		
main.py	Punto de entrada	Orquesta y ejecuta

2.2 — Prompt para que la IA refactorice

Redacta el prompt completo que le darías a la IA para que realice la refactorización. Incluye los 5 elementos del prompt profesional:

Escribe tu prompt aquí:

2.3 — Escribe el código refactorizado

Pega aquí el código de cada módulo tras la refactorización. Indica el nombre del archivo encima de cada bloque:

```
# calculadora/precios.py
# (escribe aquí tu código)
```

```
# calculadora/tokens.py
# (escribe aquí tu código)
```

```
# calculadora/proyecciones.py
# (escribe aquí tu código)
```

BLOQUE 3 — Tests Unitarios

📌 *Regla profesional: nunca se sube código a producción sin tests. En este bloque escribirás tests para cada módulo usando pytest y aprenderás a pedir a la IA que los genere correctamente.*

3.1 — Prompt para que la IA genere los tests

Escribe el prompt que usarías para que la IA te genere los tests unitarios del módulo precios.py. Recuerda pedir: casos felices, casos borde y casos de error:

Escribe tu prompt aquí:

3.2 — Escribe al menos 5 casos de test

Crea el archivo tests/test_precios.py con mínimo 5 casos de prueba. Usa la tabla para planificarlos antes de escribir el código:

Nombre del test	Tipo (feliz/borde/error)	Input / condición	Resultado esperado
`test_1_`			
`test_2_`			
`test_3_`			
`test_4_`			
`test_5_`			

3.3 — Código de los tests

```
# tests/test_precios.py
import pytest
from calculadora.precios import CalculadoraPrecios

# Escribe aquí tus tests
```

BLOQUE 4 — Pipeline CI/CD con GitHub Actions

🔗 *Conexión con el módulo de Pipeline: con el código modularizado y los tests escritos, el último paso es automatizar la validación. Cada vez que hagas git push, GitHub Actions ejecutará linter, tests y análisis de seguridad automáticamente.*

4.1 — Diseña el fichero .github/workflows/ci.yml

Indica qué debe hacer cada fase del pipeline y qué herramienta usarías:

Fase del pipeline	¿Qué debe hacer?	Comando / herramienta
1. Checkout del código		actions/checkout@v4
2. Configurar Python		actions/setup-python@v5
3. Instalar dependencias		pip install -r requirements.txt
4. Ejecutar linter		flake8 / ruff
5. Tests unitarios		pytest
6. Cobertura de tests		pytest --cov
7. Análisis de seguridad		bandit

4.2 — Escribe el fichero ci.yml completo

```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # Completa aquí cada step del pipeline
```

4.3 — ¿Qué ocurre si un test falla en el pipeline?

Explica el flujo: ¿se bloquea el merge? ¿Quién recibe la notificación? ¿Cómo se depura el error desde los logs de GitHub Actions?

Escribe tu respuesta aquí:

BLOQUE 5 — Documentación Profesional

📌 *Un proyecto sin documentación es un proyecto muerto. En este bloque crearás el README y la carpeta docs/ siguiendo estándares profesionales. La IA es tu aliada para generarla, pero tú decides qué incluir.*

5.1 — Estructura de la carpeta docs/

Define qué archivos crearás dentro de docs/ y qué contendrá cada uno:

Archivo en docs/	Contenido / propósito

5.2 — Escribe el README.md del proyecto

El README debe incluir: descripción del proyecto, instalación, uso, estructura de carpetas, cómo ejecutar los tests y badge del pipeline CI. Usa markdown:

```
# README.md

# Calculadora de Costes de APIs IA

## Descripción


## Instalación
```bash

```

Uso

Estructura del proyecto
```

```

## Tests
```bash

```

Pipeline CI
```

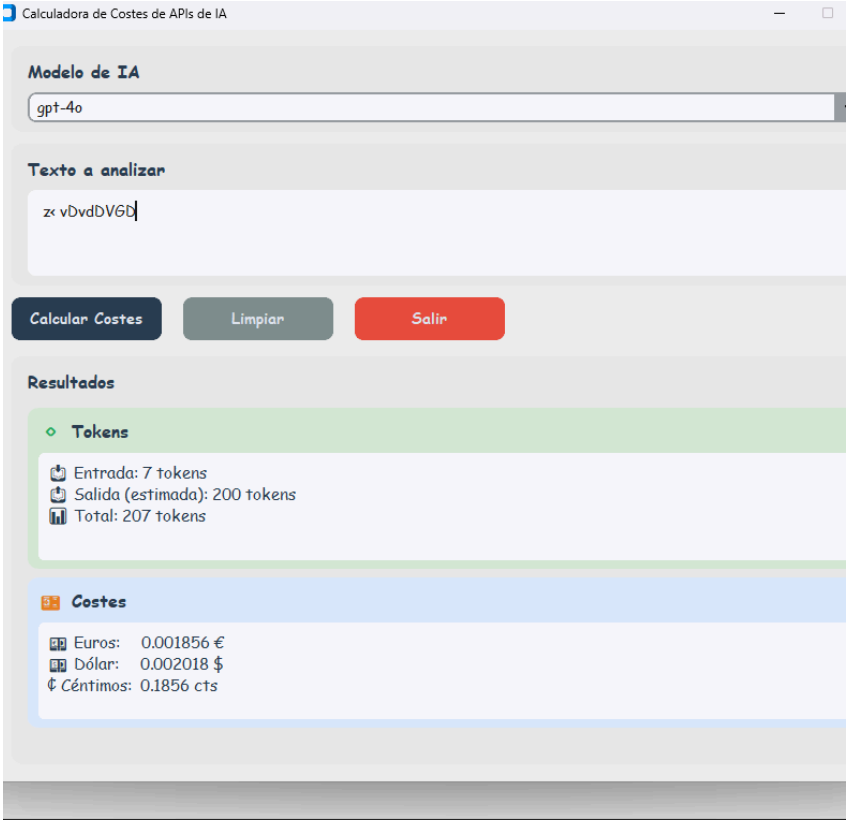
```
## Licencia
```

5.3 — Commits semánticos

Indica qué mensaje de commit escribirías para cada cambio realizado en este ejercicio. Usa el formato: tipo(scope): descripción corta

Tipo de cambio	Tu mensaje de commit	¿Por qué este mensaje?
Modularización		
Añadir tests		
CI/CD		
Documentación README		
Corrección de bug		

BLOQUE 6 — Resultado final de la PYME



BLOQUE 7 — Reflexión Integrada

Reflexión final: Describe el flujo profesional completo que acabas de implementar. ¿Qué aporta cada paso (análisis → modularización → tests → CI/CD → docs)? ¿Qué hubiera pasado si te saltas alguno en un proyecto real?

7.1 — Flujo profesional completo

Describe con tus palabras qué aprendiste en cada fase y por qué el orden importa:

Escribe tu reflexión aquí:

7.2 — ¿Cómo usaste la IA como copiloto?

Para cada bloque, indica en qué momento pediste ayuda a la IA y cómo validaste su respuesta antes de aplicarla:

Bloque	¿Para qué usé la IA?	¿Cómo verifiqué su respuesta?
Bloque 2 — Modularización		
Bloque 3 — Tests		
Bloque 4 — CI/CD		
Bloque 5 — Documentación		

7.3 — Autoevaluación

Marca con una X tu nivel de confianza al finalizar el ejercicio:

Nivel	Descripción	¿Qué necesitarías repasar?
⬜ 1	Necesito más práctica	
⬜ 2	Lo entiendo con ayuda	
⬜ 3	Lo aplico con seguridad	
⬜ 4		

BLOQUE 8 — Reflexión MemoriaFinal.md

En una carpeta de MemoriaFinal, señalar en un archivo llamado memoriaFinal.md donde explicareis el porqué el código original está mal configurado y que cambios realizasteis